

PRACTICAL WEEK-8

Task1: Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), realloc(), and free(). Also identify and analyze memory leaks using Valgrind.

```
madhulika@LAPTOP-D2M8NKBH:~$ mkdir -p ~/practical/task1
madhulika@LAPTOP-D2M8NKBH:~$ cd ~/practical/task1
madhulika@LAPTOP-D2M8NKBH:~/practical/task1$ nano memory_allocation.c
madhulika@LAPTOP-D2M8NKBH:~/practical/task1$ gcc memory_allocation.c -o memory_allocation
madhulika@LAPTOP-D2M8NKBH:~/practical/task1$ ./memory_allocation
malloc(): 10 20 30 40 50
calloc(): 0 0 0 0 0
realloc(): 10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
madhulika@LAPTOP-D2M8NKBH:~/practical/task1$ valgrind --leak-check=full ./memory_allocation
==656== Memcheck, a memory error detector
==656== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==656== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==656== Command: ./memory_allocation
==656==
malloc(): 10 20 30 40 50
calloc(): 0 0 0 0 0
realloc(): 10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
==656==
==656== HEAP SUMMARY:
==656==   in use at exit: 0 bytes in 0 blocks
==656==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==656==
==656== All heap blocks were freed -- no leaks are possible
==656==
==656== For lists of detected and suppressed errors, rerun with: -s
==656== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    int *a = malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = (i + 1) * 10;

    printf("malloc(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);

    int *b = calloc(5, sizeof(int));

    if (b == NULL) {
        printf("\ncalloc failed\n");
        free(a);
        return 1;
    }

    printf("\ncalloc(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);

    a = realloc(a, 10 * sizeof(int));

    if (a == NULL) {
        printf("\nrealloc failed\n");
        free(b);
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^N Replace

^K Cut
^U Paste

^T Execute
^J Justify

Observation: The program successfully demonstrated dynamic memory allocation using `malloc()`, `calloc()`, and `realloc()`. `malloc()` allocated memory with assigned values, while `calloc()` initialized the allocated memory to zero. `realloc()` successfully increased the allocated memory size. Finally, `free()` released the allocated memory. Valgrind was used to check memory leaks, and after properly freeing the memory, no memory leaks were detected.

Task2: Write a C program to demonstrate the Copy-on-Write (COW) mechanism after a `fork()` system call. Show that the parent and child processes have separate copies of the data when the child modifies it.

```
madhulika@LAPTOP-D2M8NKBH:~/practical/task1$ mkdir -p ~/practical/task2
madhulika@LAPTOP-D2M8NKBH:~/practical/task1$ cd ~/practical/task2
madhulika@LAPTOP-D2M8NKBH:~/practical/task2$ pwd
/home/madhulika/practical/task2
madhulika@LAPTOP-D2M8NKBH:~/practical/task2$ nano copy_on_write.c
madhulika@LAPTOP-D2M8NKBH:~/practical/task2$ gcc copy_on_write.c -o copy_on_write
madhulika@LAPTOP-D2M8NKBH:~/practical/task2$ ./copy_on_write
Before fork()
Parent PID: 905
Data: 10 20 30 40 50
Address: 0x63e72b476010

--- CHILD PROCESS ---
Child PID: 906
Before modification: 10 20 30 40 50
After modification: 999 20 30 40 50

--- PARENT PROCESS ---
Parent PID: 905
Parent data: 10 20 30 40 50
madhulika@LAPTOP-D2M8NKBH:~/practical/task2$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int *data = malloc(5 * sizeof(int));

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    // Initialize data
    for (int i = 0; i < 5; i++)
        data[i] = (i + 1) * 10;

    printf("Before fork()\n");
    printf("Parent PID: %d\n", getpid());

    printf("Data: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", data[i]);

    printf("\nAddress: %p\n", (void *)data);

    // Create child process
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

GNU nano 8.7.1 copy_on_write.c *
    for (int i = 0; i < 5; i++)
        printf("%d ", data[i]);

    printf("\n");

    // Modify data in child
    data[0] = 999;

    printf("After modification: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", data[i]);

    printf("\n");

    free(data);
}

// Parent process
else
{
    wait(NULL);

    printf("\n--- PARENT PROCESS ---\n");
    printf("Parent PID: %d\n", getpid());

    printf("Parent data: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", data[i]);

    printf("\n");

    free(data);
}

return 0;

```

Observation: The `fork()` system call created a child process. Initially, both parent and child had the same data: 10 20 30 40 50. When the child modified the first value to 999, only the child's data changed, while the parent's data remained 10 20 30 40 50. This demonstrates the Copy-on-Write (COW) mechanism, where separate memory copies are created when modification occurs.